Exponential Moving Average (EMA) Filters

Contents

1. Overview

The *exponential moving average* (EMA) filter is a **discrete, low-pass, infinite-impulse response (IIR) filter**. It **places more weight on recent data by exponentially discounting old data and behaves similarly to the** discrete first-order low-pass RC filter**.

Unlike an SMA(Simple Moving Average), most EMA filters are not windowed, and the next value depends on all previous inputs. Thus most EMA filters are ***a form of infinite impulse response (IIR) filter***, whilst an SMA is a finite impulse response (FIR) filter.

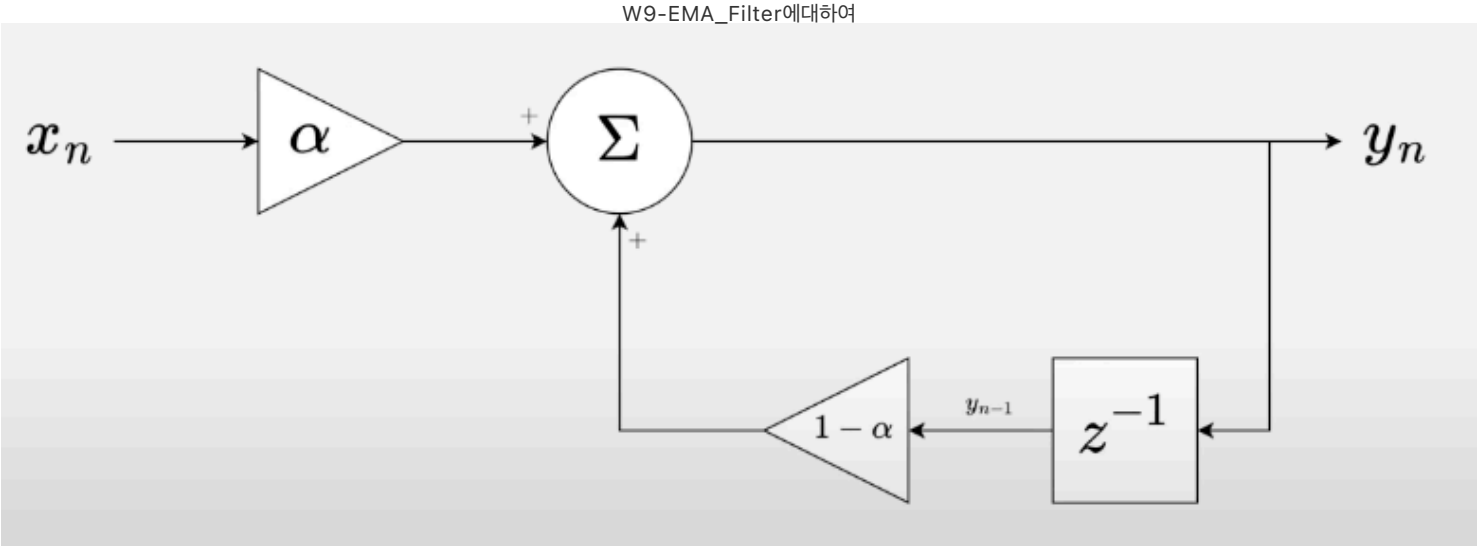
There are exceptions, and you can build a windowed exponential moving average filter where the coefficients are weighted exponentially.

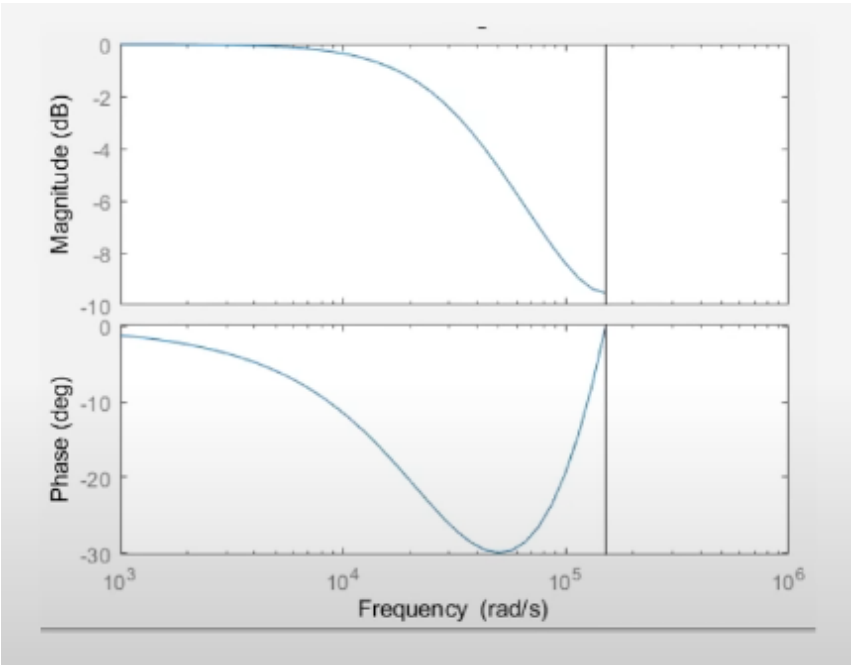
2. EMA Equation

The *difference equation* for an exponential moving average filter is:

$$y[i] = \alpha \cdot x[i] + (1 - \alpha) \cdot y[i-1]$$

where: y is the output ($[i]$ denotes the sample number), x is the input, α is a constant which sets the cutoff frequency (a value between 0 and 1)





Notice that the calculation does not require the storage of past values of x and only the previous value of y, **which makes this filter memory and computation-friendly (especially relevant for microcontrollers)**.

Only one addition, one subtraction, and two multiplication operations are needed.`

The constant α determines how aggressive the filter is. It can vary between 0 and 1 (inclusive).

As $\alpha \to 0$, the filter gets more and more aggressive, until at $\alpha = 0$, where the input does not affect the output (if the filter started like this, then the output would stay at 0).

As $\alpha \to 1$, the filter lets more of the raw input through at less filtered data, until at $\alpha = 1$, where the filter is not "filtering" at all (pass-through from input to output).`

The filter is called ***exponential*** because the *weighted contribution of previous inputs decreases exponentially the further the input is away in time*.

This can be seen in the difference equation if we substitute in previous inputs:

$$
\begin{aligned}
y[i] &= \alpha \cdot x[i] + (1-\alpha) \cdot y[i-1] \\
&= \alpha \cdot x[i] + (1-\alpha) \cdot [\alpha \cdot x[i-1] + (1-\alpha) \cdot y[i-2]] \\
&= \alpha \cdot x[i] + (1-\alpha) \cdot [\alpha \cdot x[i-1] + (1-\alpha) \cdot [\alpha \cdot x[i-2] + (1-\alpha) \cdot y[i-3]]] \\
&= \ldots \\
&= \alpha \sum_{k=0}^{n} (1-\alpha)^k x[n-k] \qquad (2)
\end{aligned}
$$

The following code implements an IIR EMA filter in C++, suitable for microcontrollers and other embedded devices.

Fixed-point numbers are used instead of floats to speed up computation. `K` is the number of fractional bits used in the fixed-point representation.

```
template <uint8_t K, class uint_t = uint16_t>
class EMA {
  public:
    /// Update the filter with the given input and return the filtered output.
    uint_t operator()(uint_t input) {
        state += input;
        uint_t output = (state + half) >> K;
        state -= output;
        return output;
    }

    static_assert(
        uint_t(0) < uint_t(-1),  // Check that `uint_t` is an unsigned type
        "The `uint_t` type should be an unsigned integer, otherwise, "
        "the division using bit shifts is invalid.");

    /// Fixed point representation of one half, used for rounding.
    constexpr static uint_t half = 1 << (K - 1);

  private:
    uint_t state = 0;
};
```

3. Frequency Response

The frequency response of the EMA filter can be found by using the Z transform. If we start with the time-domain equation for an EMA filter:

$$y[i] = \alpha \cdot x[i] + (1 - \alpha) \cdot y[i - 1] \tag{3}$$

And then take the Z transform of it:

$$Y(z) = \alpha X(z) + (1 - \alpha) z^{-1} Y(z) \tag{4}$$

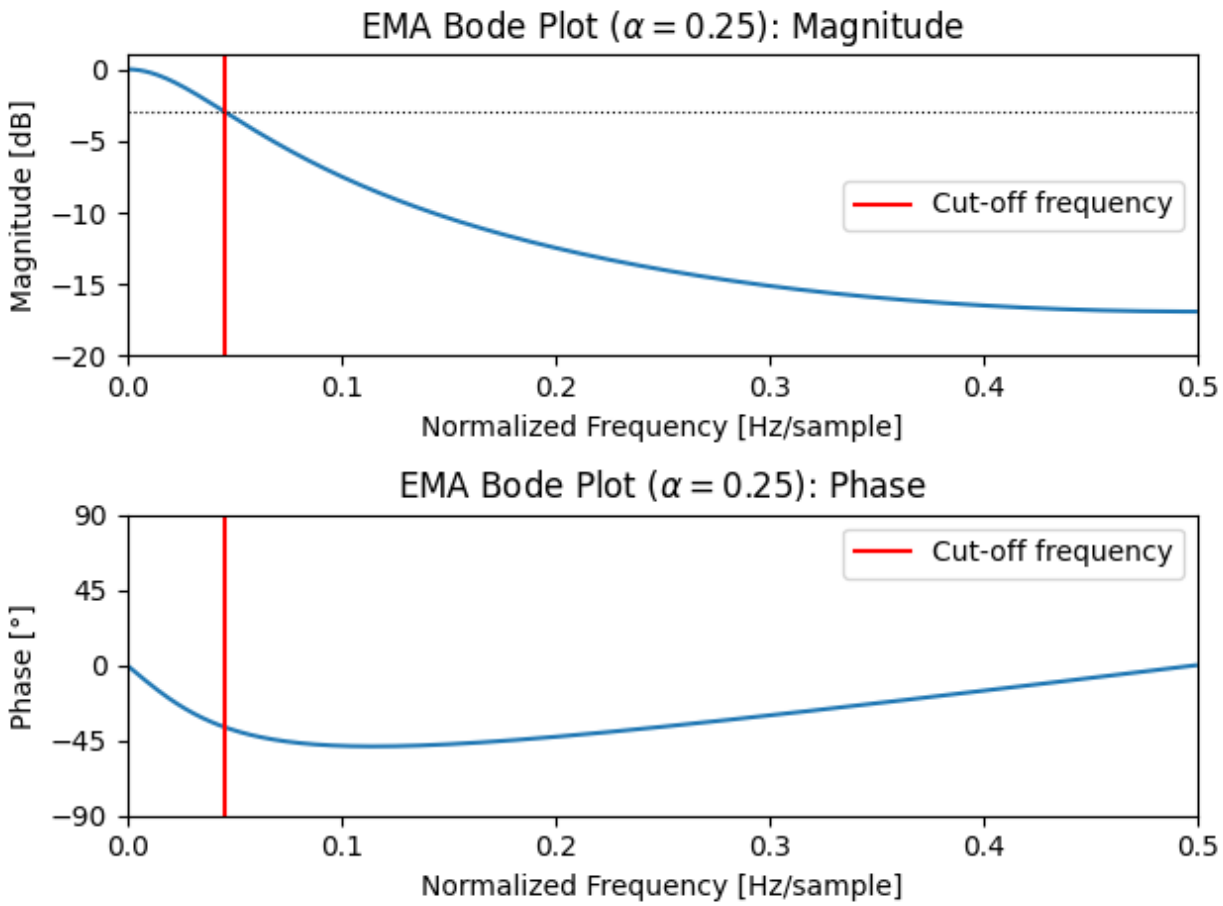
Then re-arrange to find the transfer function `H(z):`

$$\begin{aligned} H(z) &= Y(z)X(z) \\ &= \frac{\alpha}{1 - (1 - \alpha)z^{-1}} \\ &= \frac{\alpha z}{z - (1 - \alpha)} \end{aligned} \tag{5}$$

This transfer function can be used to create bode plots of the magnitude and phase response of the EMA filter.

The below bode plot shows the response of an EMA filter with $\alpha = 0.25$.

The x-axis frequency is the normalized frequency, in units $Hz/sample$, which makes the plot applicable for any sampling frequency.



Bode plot showing the magnitude and phase of an EMA filter with ($\alpha = 0.25$).

```
from scipy.signal import freqz
import matplotlib.pyplot as plt
from math import pi, acos
import numpy as np

alpha = 0.25

b = np.array(alpha)
a = np.array((1, alpha - 1))

print("b =", b)                         # Print the coefficients
print("a =", a)

x = (alpha**2 + 2*alpha - 2) / (2*alpha - 2)
w_c = acos(x)                           # Calculate the cut-off frequency

w, h = freqz(b, a)                      # Calculate the frequency response

plt.subplot(2, 1, 1)                    # Plot the amplitude response
plt.suptitle('Bode Plot')
plt.plot(w, 20 * np.log10(abs(h)))      # Convert to dB
plt.ylabel('Magnitude [dB]')
plt.xlim(0, pi)
plt.ylim(-18, 1)
plt.axvline(w_c, color='red')
plt.axhline(-3, linewidth=0.8, color='black', linestyle=':')

plt.subplot(2, 1, 2)                    # Plot the phase response
plt.plot(w, 180 * np.angle(h) / pi)     # Convert argument to degrees
```

```
plt.xlabel('Frequency [rad/sample]')
plt.ylabel('Phase [°]')
plt.xlim(0, pi)
plt.ylim(-90, 90)
plt.yticks([-90, -45, 0, 45, 90])
plt.axvline(w_c, color='red')
plt.show()
```

The *cut-off frequency* (-3dB point) of an EMA filter is given by:

$$f_c = \frac{f_s}{2\pi} cos^{-1}\left[1 - \frac{\alpha^2}{2(1-\alpha)}\right] \tag{6}$$

where f_s is the sampling frequency in Hz.

The `cutoff frequency` is defined as the frequency of the half-power point, where the power gain is a half.

It's often called the -3dB point, because $10\log_{10}(\frac{1}{2}) \approx -3.01\ dB$.
To find it, just solve the following equation:

$$\frac{\left|H\left(e^{j\omega_c}\right)\right|^2}{\frac{\alpha^2}{1-2(1-\alpha)\cos(\omega_c)+(1+\alpha)^2}} = \frac{1}{2}$$
$$\omega_c = \cos^{-1}\left(\frac{\alpha^2+2\alpha-2}{2\alpha-2}\right)$$

For example, if $ \alpha =0.25$, then

$$\omega_c = \cos^{-1}(\frac{23}{24}) \approx 0.2897 \frac{rad}{sample}$$

To convert it to a frequency in `Hertz`, you can multiply ω by $\frac{f_s}{2\pi}$, with f_s the sample frequency.
For example, if the sample frequency is

$$f_s = 1000\frac{samples}{s},\ \text{then}\ f = \cos^{-1}(\frac{23}{24})\frac{f_s}{2\pi} \approx 46.12Hz$$

3.1 Plotting the frequency response in Python

We can use the SciPy and Matplotlib modules to plot the frequency response in Python.

The SciPy `freqz` function expects the transfer function coefficients in the form

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + \cdots + b_p z^{-p}}{a_0 + a_1 z^{-1} + a_2 z^{-2} + \cdots + a_q z^{-q}}$$

This is the reverse of the usual ordering of polynomial coefficients.

In the case of the exponential moving average filter, the transfer function is

$$H(z) = \frac{\alpha}{1 + (\alpha - 1)z^{-1}}$$

so, $b_0 = \alpha,\ a_0 = 1$ and $a_1 = \alpha - 1$

4. Impulse Response

The discrete unit sample function is defined as:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases} \tag{7}$$

If we use this as our input into `Eq. 2:`

$$y[i] = \alpha \sum_{k=0}^{n} (1-\alpha)^k \delta[n-k] \tag{8}$$

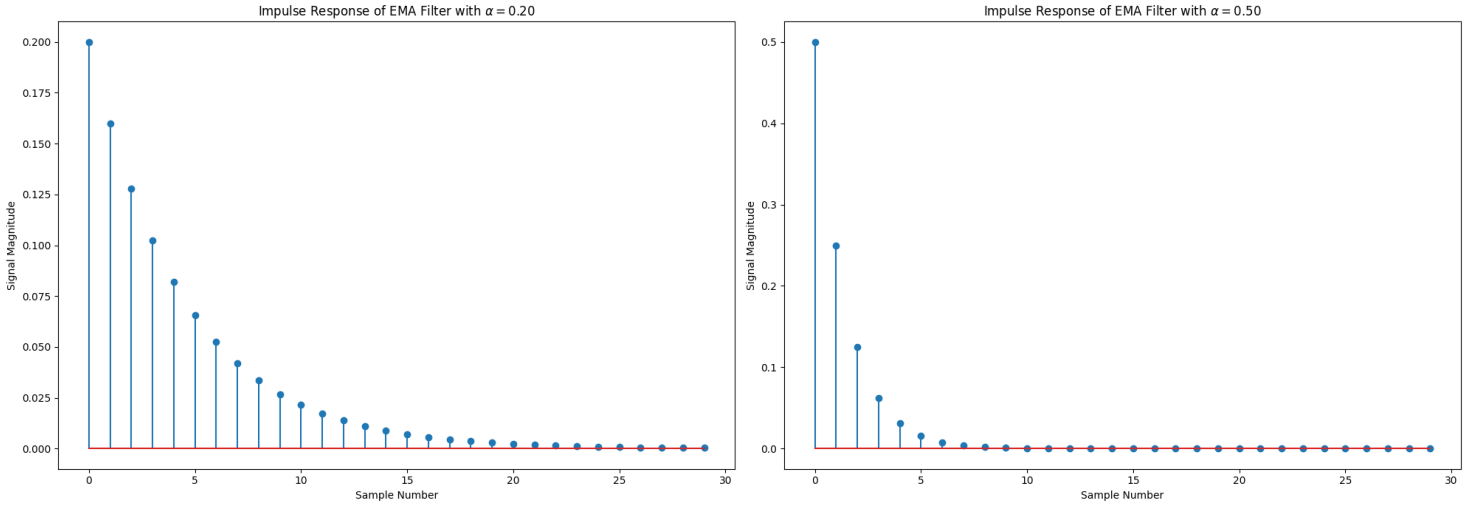
Given the unit sample function is 0 at most points, the only sum term that matters is when $k = n$, so we can simplify this to:

$$y[i] = \alpha(1-\alpha)^n \tag{9}$$

From this, we can plot what the response will look like for impulse as the input.

As you can see in the following graph, the output starts off at $y[0] = \alpha$ and then decays towards 0.

A larger alpha makes the initial response larger but also the decay faster.`



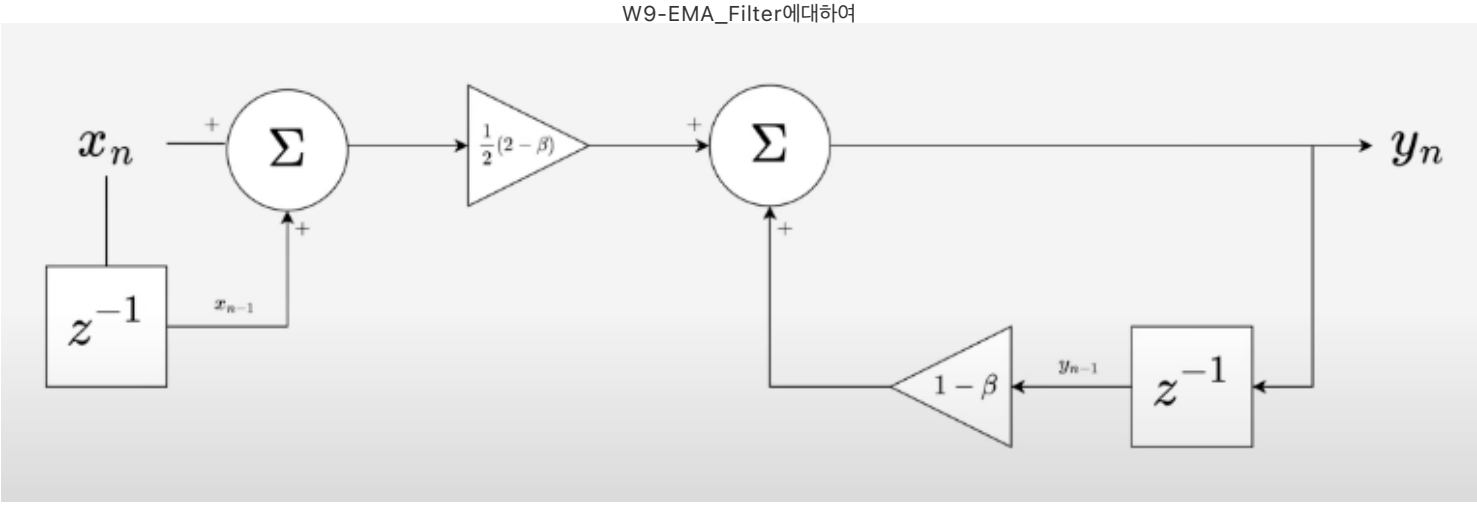
Impulse response for an EMA filter with different (α) values.

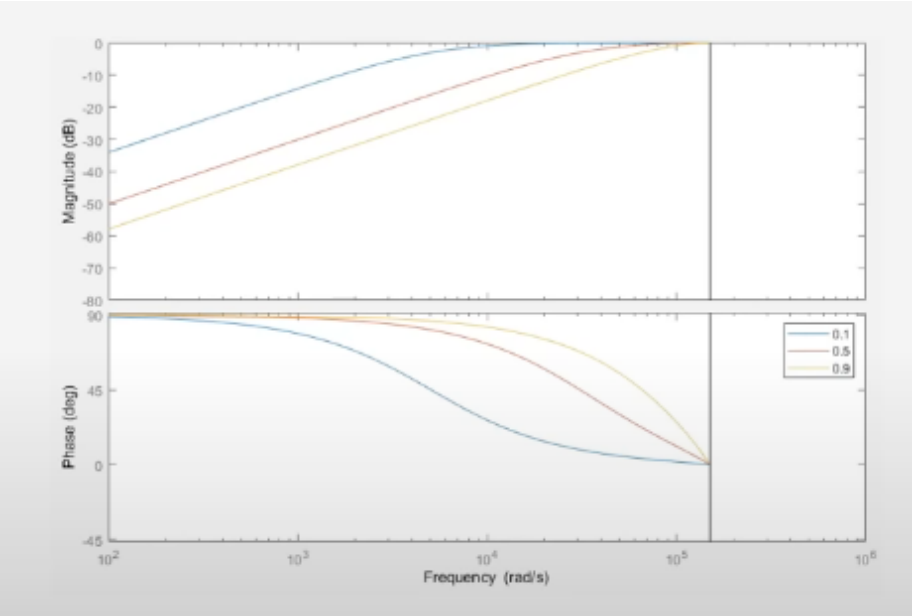
5. EMA High-pass filter

The *difference equation* for an exponential moving average(EMA) HPF filter is:

$$y[i] = \frac{1}{2} \cdot (2-\beta) \cdot (x[i] - x[i-1]) + (1-\beta) \cdot y[i-1]$$

where: y is the output ($[i]$ denotes the sample number), x is the input, β is constant which sets the cutoff frequency (a value between 0 and 1)





6. EMA HPF Frequency Response

If we start with the time-domain equation for an EMA HPF filter:

$$y[i] = \frac{1}{2} \cdot (2-\beta) \cdot (x[i] - x[i-1]) + (1-\beta) \cdot y[i-1]$$

And then take the Z transform of it:

$$\begin{aligned} Y(z) &= (1-\frac{\beta}{2}) \cdot (X(z) - z^{-1}X(z) + (1-\beta) \cdot z^{-1}Y(z) \\ &= (1-\frac{\beta}{2}) \cdot (1-z^{-1})X(z) + (1-\beta) \cdot z^{-1}Y(z) \\ (1+(\beta-1) \cdot z^{-1})Y(z) &= (1-\frac{\beta}{2}) \cdot (1-z^{-1})X(z) \end{aligned}$$

Then re-arrange to find the transfer function `H(z):`

$$\begin{aligned} H(z) = \frac{Y(z)}{X(z)} &= (1-\frac{\beta}{2}) \cdot \frac{(1-z^{-1})}{(1+(\beta-1) \cdot z^{-1})} \\ &= (1-\frac{\beta}{2}) \cdot \frac{(z-1)}{(z+(\beta-1))} \end{aligned}$$

This transfer function can be used to create bode plots of the magnitude and phase response of the EMA filter.

⚙️ EMA-LPF & EMA-HPF — STM32CubeIDE C 함수

📐 필터 계수 계산

조건

- **샘플링 주파수**: $f_s = 100\ \text{Hz}$
- **컷오프 주파수**: $f_c = 20\ \text{Hz}$

α (LPF), β (HPF) 계산

$$\alpha = \beta = \frac{2\pi f_c}{2\pi f_c + f_s} = \frac{2\pi \times 20}{2\pi \times 20 + 100} \approx 0.557$$

📁 헤더 파일 — `ema_filter.h`

```c
/* ema_filter.h */
#ifndef EMA_FILTER_H
#define EMA_FILTER_H

#include <stdint.h>

/* ──────────────────────────────────────────
 * 필터 설정값
 * fs = 100Hz, fc = 20Hz
 * alpha = beta = 2*pi*fc / (2*pi*fc + fs)
 * ────────────────────────────────────────── */
#define EMA_ALPHA   (0.557f)   /* LPF 계수 */
#define EMA_BETA    (0.557f)   /* HPF 계수 */

/* ──────────────────────────────────────────
 * LPF 상태 구조체
 * ────────────────────────────────────────── */
typedef struct {
    float y_prev;   /* 이전 출력값 y[i-1] */
    uint8_t init;   /* 초기화 플래그 */
} EMA_LPF_State;

/* ──────────────────────────────────────────
 * HPF 상태 구조체
 * ────────────────────────────────────────── */
typedef struct {
    float x_prev;   /* 이전 입력값 x[i-1] */
    float y_prev;   /* 이전 출력값 y[i-1] */
    uint8_t init;   /* 초기화 플래그 */
} EMA_HPF_State;

/* ──────────────────────────────────────────
 * 함수 프로토타입
 * ────────────────────────────────────────── */
void  EMA_LPF_Init(EMA_LPF_State *state);
float EMA_LPF_Update(EMA_LPF_State *state, float x);
```

```
void  EMA_HPF_Init(EMA_HPF_State *state);
float EMA_HPF_Update(EMA_HPF_State *state, float x);

#endif /* EMA_FILTER_H */
```

📁 소스 파일 — `ema_filter.c`

```
/* ema_filter.c */
#include "ema_filter.h"

/* ═══════════════════════════════════════════════
 *  EMA Low-Pass Filter (저역 통과 필터)
 *
 *  차분 방정식:
 *    y[i] = α * x[i] + (1 - α) * y[i-1]
 *
 *  전달 함수:
 *          α * z
 *  H(z) = ──────────
 *          z - (1 - α)
 *
 *  fs=100Hz, fc=20Hz → α ≈ 0.557
 * ═══════════════════════════════════════════════ */

/**
 * @brief  EMA LPF 상태 초기화
 * @param  state: LPF 상태 구조체 포인터
 */
void EMA_LPF_Init(EMA_LPF_State *state)
{
    state->y_prev = 0.0f;
    state->init   = 0;
}

/**
 * @brief  EMA LPF 업데이트 (매 샘플마다 호출)
 * @param  state: LPF 상태 구조체 포인터
 * @param  x    : 현재 입력값 x[i]
 * @retval float: 필터링된 출력값 y[i]
 */
float EMA_LPF_Update(EMA_LPF_State *state, float x)
{
    float y;

    /* 첫 샘플은 그대로 출력 (과도응답 방지) */
    if (state->init == 0) {
        state->y_prev = x;
        state->init   = 1;
        return x;
    }

    /* y[i] = α * x[i] + (1 - α) * y[i-1] */
    y = EMA_ALPHA * x + (1.0f - EMA_ALPHA) * state->y_prev;

    state->y_prev = y;
```

```
    return y;
}


/* ═══════════════════════════════════════════════════
 *  EMA High-Pass Filter (고역 통과 필터)
 *
 *  차분 방정식:
 *    y[i] = (1 - β/2) * (x[i] - x[i-1]) + (1 - β) * y[i-1]
 *
 *  전달 함수:
 *              z - 1
 *  H(z) = (1 - β/2) * ──────────────
 *                      z - (1 - β)
 *
 *  Zero : z = +1   → DC(0Hz) 완전 차단
 *  Pole : z = 1-β → 고주파 통과
 *
 *  fs=100Hz, fc=20Hz → β ≈ 0.557
 * ═══════════════════════════════════════════════════ */

/**
 * @brief  EMA HPF 상태 초기화
 * @param  state: HPF 상태 구조체 포인터
 */
void EMA_HPF_Init(EMA_HPF_State *state)
{
    state->x_prev = 0.0f;
    state->y_prev = 0.0f;
    state->init   = 0;
}

/**
 * @brief  EMA HPF 업데이트 (매 샘플마다 호출)
 * @param  state: HPF 상태 구조체 포인터
 * @param  x    : 현재 입력값 x[i]
 * @retval float: 필터링된 출력값 y[i]
 */
float EMA_HPF_Update(EMA_HPF_State *state, float x)
{
    float y;

    /* 첫 샘플은 0 출력 (초기 차분값 없음) */
    if (state->init == 0) {
        state->x_prev = x;
        state->y_prev = 0.0f;
        state->init   = 1;
        return 0.0f;
    }

    /* y[i] = (1 - β/2) * (x[i] - x[i-1]) + (1 - β) * y[i-1] */
    y = (1.0f - EMA_BETA / 2.0f) * (x - state->x_prev)
        + (1.0f - EMA_BETA) * state->y_prev;

    state->x_prev = x;
    state->y_prev = y;
    return y;
}
```

📋 사용 예시 — `main.c`

```c
#include "ema_filter.h"

/* 전역 상태 변수 선언 */
EMA_LPF_State lpf;
EMA_HPF_State hpf;

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    /* 필터 초기화 */
    EMA_LPF_Init(&lpf);
    EMA_HPF_Init(&hpf);

    float raw_signal   = 0.0f;
    float lpf_output   = 0.0f;
    float hpf_output   = 0.0f;

    while (1)
    {
        /* 10ms마다 실행 (100Hz 샘플링) */
        HAL_Delay(10);

        /* 센서에서 원시 데이터 획득 (예시) */
        raw_signal = Read_Sensor();

        /* LPF 적용 */
        lpf_output = EMA_LPF_Update(&lpf, raw_signal);

        /* HPF 적용 */
        hpf_output = EMA_HPF_Update(&hpf, raw_signal);
    }
}
```

📊 계수 요약표

파라미터	값	설명
f_s	100 Hz	샘플링 주파수
f_c	20 Hz	컷오프 주파수
α	**0.557**	LPF 계수
β	**0.557**	HPF 계수
LPF 극점	$z = 0.443$	저주파 통과
HPF 영점	$z = +1$	DC 완전 차단
HPF 극점	$z = 0.443$	고주파 통과

> 💡 **TIP**: 타이머 인터럽트(TIMx)를 사용해 정확히 **10ms(100Hz)** 주기로 `EMA_LPF_Update()` / `EMA_HPF_Update()`를 호출하면 더욱 정밀한 필터링이 가능합니다!